

BOOKMAP TRAPS INDICATOR REFERENCE

```
// This Pine Script® code is subject to the terms of the Mozilla Public License 2.0 at https://mozilla.org/MPL/2.0/
// © drop_trades

//@version=6
indicator("BIG TRAP", "Trap", overlay=true, max_labels_count=500)

// --- 1. Settings ---
group_general = "General Settings"
ticksPerRow   = input.int(5, "Ticks per Row", minval=1, group=group_general, tooltip="Aggregates price ticks. 1 = detailed, 5+ = clustered")
showPOC       = input.bool(false, "Show POC Line", group=group_general)
maxLookback   = input.int(500, "Max Lookback (Bars)", minval=1, group=group_general, tooltip="Limit calculation to the most recent X bars. Footprint data accuracy degrades significantly on deep historical bars.")

group_filters = "Bubble Filters"
imbalanceRatio = input.float(3.0, "Imbalance Ratio", minval=1.0, step=0.1, group=group_filters, tooltip="Buy Vol must be X times Sell Vol (or vice versa)")
minVolFilter   = input.int(30, "Min Trap Volume (Total)", minval=1, group=group_filters, tooltip="Total aggregated trapped volume required to spawn the bubble.")
minDeltaFilter = input.int(0, "Min Delta Filter (Per Row)", minval=0, group=group_filters, tooltip="Individual rows must meet this delta to be included in the sum.")

// --- TRAP LOGIC SETTINGS ---
group_traps = "Trap Detection Strategy"
showTrapsOnly = input.bool(true, "Show TRAPS Only", group=group_traps, tooltip="If checked, regular imbalances are hidden. Only shows 'Underwater' imbalances.")
trapStrict    = input.bool(true, "Strict Mode (Wicks Only)", group=group_traps, tooltip="Only count traps if they occur in the top/bottom 30% of the candle.")
topPctFilter  = input.float(15, "Show Top % of Traps", minval=1.0, maxval=100.0, step=1.0, group=group_traps, tooltip="Filter to only show the largest traps. e.g., 10% will only show the top 10% biggest volume traps.")

// --- VISUAL SETTINGS (MAIN) ---
group_vis_main = "Visual Settings: Main"
```

```

volVisualMode = input.string("Size Scaling", "Volume Indicator Mode", options=["Size Scaling",
"Transparency Scaling"], group=group_vis_main, tooltip="Choose whether volume is shown by
changing the Size or the Transparency.")
autoScale     = input.bool(true, "Auto-Scale (Smart)", group=group_vis_main, tooltip="If ON, the
script calculates the max volume based on current chart data.")
outlierPct    = input.float(90.0, "Outlier Percentile Cap", minval=50.0, maxval=100.0, step=5.0,
group=group_vis_main, tooltip="The volume percentile that counts as 'Huge'. 90 means the top
10% of bubbles are max size. Used for BOTH Size and Transparency modes.")
colBull       = input×color(color×new(color×teal, 10), "Trapped Buyers Color",
group=group_vis_main)
colBear       = input×color(color×new(color×red, 10), "Trapped Sellers Color",
group=group_vis_main)

// --- VISUAL SETTINGS (SUB-GROUPS) ---
group_vis_size = "Visual Settings: Size Mode Details"
maxVolHuge     = input×int(1000, "Manual Max Volume", minval=1, group=group_vis_size,
tooltip="Only used if Auto-Scale is OFF.")
sizeFixedTransp = input.int(40, "Fixed Transparency", minval=0, maxval=100,
group=group_vis_size, tooltip="Transparency used when Size Scaling is active.")

group_vis_trans = "Visual Settings: Transparency Mode Details"
transpFixedSize = input.string(size.small, "Fixed Bubble Size", options=[size.tiny, size.small,
size.normal, size.large, size.huge], group=group_vis_trans, tooltip="Size used when Transparency
Scaling is active.")
transpStart     = input.int(80, "Start Transparency", minval=0, maxval=100, group=group_vis_trans,
tooltip="Very transparent for low volume.")
transpEnd       = input.int(10, "End Transparency", minval=0, maxval=100, group=group_vis_trans,
tooltip="Solid/Opaque for high volume.")

// --- 2. State & Cache ---
var label[] cache_lbl   = array.new_label()
var float[] cache_vol   = array.new_float()
var bool[]  cache_is_bull = array.new_bool()

// Request Footprint Data
footprintData = request×footprint(ticksPerRow, 70, 300)

// --- 3. Helper Functions ---

```

```

getPrice(rowObj) =>
    na(rowObj) ? na : volume_row.down_price(rowObj)

calcSize(vol, minV, maxV) =>
    safeMax = maxV > minV ? maxV : minV + 100
    rangeV = safeMax - minV
    sz = size.tiny

    // Thresholds relative to the Percentile Max
    if vol >= minV + (rangeV * 0.15)
        sz := size.small
    if vol >= minV + (rangeV * 0.40)
        sz := size.normal
    if vol >= minV + (rangeV * 0.70)
        sz := size.large
    // Anything above the Percentile Max is Huge
    if vol >= safeMax
        sz := size.huge
    sz

calcTransp(vol, minV, maxV, tStart, tEnd) =>
    safeMax = maxV > minV ? maxV : minV + 100
    rangeV = safeMax - minV
    strength = (vol - minV) / rangeV
    strength := math.max(0.0, math.min(strength, 1.0))
    tDiff = tStart - tEnd
    int(tStart - (tDiff * strength))

// --- 4. Main Aggregation Logic ---
// Check if current bar is within the user-defined lookback window
isWithinLookback = (last_bar_index - bar_index) <= maxLookback

if isWithinLookback and not na(footprintData) and array.size(footprint.rows(footprintData)) > 0

    rowsArray = footprintxrows(footprintData)
    int rowCount = arrayxsize(rowsArray)

    float barRange = high - low

```

```
float wickThreshold = barRange * 0.3 // 30% threshold for wicks
```

```
// --- Aggregation Variables ---
```

```
float aggTrapBuyVol = 0.0
```

```
float aggTrapSellVol = 0.0
```

```
float wBuyPriceSum = 0.0 // Weighted Sum for Buyers
```

```
float wSellPriceSum = 0.0 // Weighted Sum for Sellers
```

```
for i = 0 to rowCount - 1
```

```
    volume_row row = arrayxget(rowsArray, i)
```

```
    float buyVol = volume_row.buy_volume(row)
```

```
    float sellVol = volume_row.sell_volume(row)
```

```
    float price = getPrice(row)
```

```
    float totalRowVol = buyVol + sellVol
```

```
    float netDelta = buyVol - sellVol
```

```
    float absDelta = math.abs(netDelta)
```

```
    // Check Row Qualification
```

```
    if absDelta >= minDeltaFilter
```

```
        float buyRatio = sellVol > 0 ? buyVol / sellVol : buyVol
```

```
        float sellRatio = buyVol > 0 ? sellVol / buyVol : sellVol
```

```
        bool isBuyImbalance = buyRatio >= imbalanceRatio
```

```
        bool isSellImbalance = sellRatio >= imbalanceRatio
```

```
    // --- TRAP LOGIC ---
```

```
    // Trapped Buyer: Imbalance AND Price > Close
```

```
    if isBuyImbalance and (price > close)
```

```
        // Strict Filter Check
```

```
        if not trapStrict or (price > low + (barRange * 0.7))
```

```
            aggTrapBuyVol := aggTrapBuyVol + totalRowVol
```

```
            wBuyPriceSum := wBuyPriceSum + (price * totalRowVol)
```

```
    // Trapped Seller: Imbalance AND Price < Close
```

```
    if isSellImbalance and (price < close)
```

```
// Strict Filter Check
if not trapStrict or (price < low + wickThreshold)
    aggTrapSellVol := aggTrapSellVol + totalRowVol
    wSellPriceSum := wSellPriceSum + (price * totalRowVol)
```

```
// --- PLOT AGGREGATED BUBBLES ---
```

```
// 1. Plot Buyer Trap (if exists)
```

```
if aggTrapBuyVol >= minVolFilter
    float avgPrice = wBuyPriceSum / aggTrapBuyVol
    lbl = label.new(
        x=bar_index,
        y=avgPrice,
        text="",
        style=label.style_circle,
        color=color.new(colBull, sizeFixedTransp),
        size=size.tiny,
        tooltip="TRAPPED BUYERS\nTotal Vol: " + str.toString(aggTrapBuyVol)
    )
    array.push(cache_lbl, lbl)
    array.push(cache_vol, aggTrapBuyVol)
    array.push(cache_is_bull, true)
```

```
// 2. Plot Seller Trap (if exists)
```

```
if aggTrapSellVol >= minVolFilter
    float avgPrice = wSellPriceSum / aggTrapSellVol
    lbl = label.new(
        x=bar_index,
        y=avgPrice,
        text="",
        style=label.style_circle,
        color=color.new(colBear, sizeFixedTransp),
        size=size.tiny,
        tooltip="TRAPPED SELLERS\nTotal Vol: " + str.toString(aggTrapSellVol)
    )
    array.push(cache_lbl, lbl)
    array.push(cache_vol, aggTrapSellVol)
    array.push(cache_is_bull, false)
```

```

// Maintain Cache Limit
while array.size(cache_lbl) > 500
    oldLbl = array.shift(cache_lbl)
    label.delete(oldLbl)
    array.shift(cache_vol)
    array.shift(cache_is_bull)

// --- 5. VISUAL UPDATE (Percentile Scaling & Top % Filter) ---
if array.size(cache_vol) > 0

    float currentMax = maxVolHuge // Default Manual Max

    // Calculate Percentiles
    if autoScale or volVisualMode == "Transparency Scaling"
        currentMax := array.percentile_linear_interpolation(cache_vol, outlierPct)
        if currentMax < minVolFilter
            currentMax := minVolFilter + 100

    // Logic for showing ONLY Top X%
    float visibilityThreshold = array.percentile_linear_interpolation(cache_vol, 100.0 - topPctFilter)

    for i = 0 to array.size(cache_lbl) - 1
        lblVal = array.get(cache_lbl, i)
        volVal = array.get(cache_vol, i)
        isBull = array.get(cache_is_bull, i)

        // Check Top % Visibility
        if volVal < visibilityThreshold
            label.set_color(lblVal, color.new(color.white, 100)) // Hide
            label.set_size(lblVal, size.tiny)
        else
            // Normal Visual Scaling Logic
            if volVisualMode == "Size Scaling"
                newSz = calcSize(volVal, minVolFilter, currentMax)
                label.set_size(lblVal, newSz)
                label.set_color(lblVal, color.new(isBull ? colBull : colBear, sizeFixedTransp))
            else

```

```
newTransp = calcTransp(volVal, minVolFilter, currentMax, transpStart, transpEnd)
label.set_color(lblVal, color.new(isBull ? colBull : colBear, newTransp))
label.set_size(lblVal, transpFixedSize)
```